Algorithm: Minimum Sum Partition

The problem asks us to divide an array into two subsets, S_1 and S_2, such that $|S_1 - S_2|$ is minimized. This is a variation of the classic **0/1 Knapsack** problem.

1. **Calculate Total Sum:** Find the sum of all elements in the array, let's call it totalSum.
2. **Define the Target:** The ideal scenario is dividing the array into two exactly equal halves, meaning each subset would sum to totalSum / 2. We will try to find a subset S_1 whose sum is as close to totalSum / 2 as possible.
3. **Initialize DP Array:** Create a 1D boolean array dp of size (totalSum / 2) + 1, initialized to false.
    - dp[j] will be true if a subset with sum j is possible.
    - Set dp[0] = true (a sum of 0 is always possible with an empty subset).
4. **Populate DP Table:** For each number num in the array:
    - Iterate backwards through the dp array from totalSum / 2 down to num.
    - Update dp[j] = dp[j] || dp[j - num]. (A sum j is possible if it was already possible, or if the sum j - num was possible before adding the current num).
5. **Find Minimum Difference:** Find the largest index j where dp[j] is true. This represents the largest possible sum for subset S_1.
6. **Calculate Result:** The sum of the other subset S_2 will be totalSum - j. The absolute difference is (totalSum - j) - j = totalSum - 2 * j.